

Centro Formativo Master D Curso de Full Stack Web Development

CASO PRÁTICO: BASES DE DADOS MYSQL
Sistema Relacional de Gestão de Eventos e Venda
de Bilhetes

Formanda: Elaine Gonçalves dos Santos **Data:** 15 de Junho de 2026.

1. Introdução

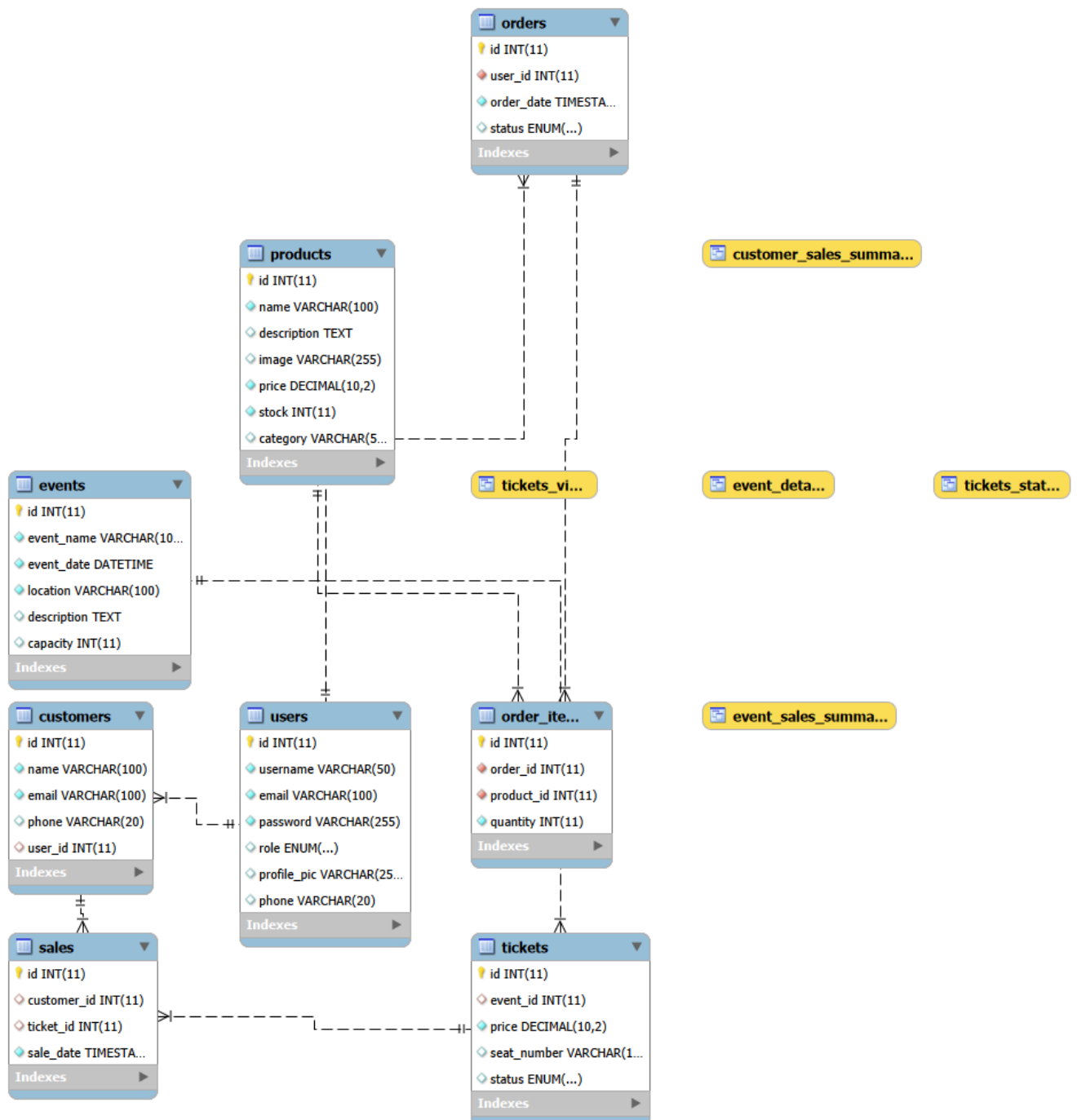
O presente documento detalha o desenvolvimento e a implementação da base de dados relacional em MySQL desenhada para a plataforma oficial da **Banda Kid Abelha**. O projeto consiste num sistema integrado que gere não só a loja de produtos da banda, mas também, e com especial foco neste caso prático, os **eventos de concertos e a respetiva venda de bilhetes**. O objetivo central é aplicar e consolidar os conceitos de modelagem de dados, garantindo a integridade referencial, a eficiência nas consultas e a segurança das transações.

A arquitetura do sistema foi projetada para responder a cenários reais de comércio eletrónico e bilheteira, suportando compras tanto para fãs com conta registada como para clientes convidados (guest checkout). O esquema relacional assenta em quatro entidades principais obrigatórias — Eventos, Bilhetes, Clientes e Vendas — perfeitamente integradas com a tabela de utilizadores pré-existentes.

Para além da estruturação das tabelas com as respectivas Chaves Primárias (PK) e Chaves Estrangeiras (FK), foram desenvolvidas Views específicas em linguagem SQL. Estas vistas lógicas permitem à administração do sistema extrair relatórios consolidados em tempo real sobre a disponibilidade de bilhetes, receitas geradas por cada concerto e o histórico de compras por fã, simplificando a consulta de dados sem comprometer a performance da aplicação.

2. Esquema Visual da Base de Dados (Diagrama Entidade-Relacionamento)

Abaixo apresenta-se o modelo físico da base de dados desenvolvido para o projeto. O diagrama ilustra de forma clara a estrutura das tabelas, os tipos de dados dos atributos e as linhas de relacionamento que unem as diferentes entidades do sistema.



3. Dicionário de Dados (Estrutura das Tabelas)

A base de dados foi estruturada de forma a normalizar a informação, evitando redundâncias e garantindo a integridade dos registos de vendas. Abaixo detalha-se o propósito de cada tabela do núcleo de bilheteira e as suas colunas principais:

3.1. Tabela **events**

Propósito: Armazenar a informação mestre sobre os concertos da Banda Kid Abelha.

- **id (PK):** Identificador único e sequencial do evento.
- **event_name:** Nome ou título do concerto/tour.
- **description:** Detalhes adicionais sobre o concerto.
- **event_date:** Regista a data e a hora agendadas para o espetáculo.
- **location:** Local físico/estádio onde o evento se realiza.
- **capacity:** Lotação máxima do recinto.

3.2. Tabela **tickets**

Propósito: Gere o inventário de bilhetes gerados para cada concerto.

- **id (PK):** Identificador único do bilhete.
- **event_id (FK):** Chave estrangeira que relaciona o bilhete ao seu respectivo concerto (liga à tabela **events**).
- **price:** Valor facial do bilhete.
- **seat_number:** Identificação do lugar marcado (se aplicável).
- **status:** Estado atual do bilhete, permitindo os valores 'Available' (Disponível) ou 'Sold' (Vendido).

3.3. Tabela **customers**

Propósito: Armazenar os dados de faturação e contacto dos compradores. O sistema está preparado para gerir clientes registados (fãs logados) e não registados (convidados).

- **id (PK):** Identificador único do cliente.
- **name:** Nome completo do comprador.
- **email:** Endereço de correio eletrónico para envio de comprovativos.
- **phone:** Contacto telefónico (obrigatório para convidados).
- **user_id (FK):** Chave estrangeira que faz a ponte com a tabela **users** (sistema de autenticação). Permite valor Nulo (NULL) caso a compra seja efetuada por um convidado sem conta.

3.4. Tabela **sales**

Propósito: Tabela central de transações. Relaciona um bilhete físico, o cliente que o comprou e o momento temporal da transação.

- **id (PK):** Identificador único da transação de venda.
- **ticket_id (FK):** Chave estrangeira que identifica qual o bilhete específico que foi transacionado (liga à tabela **tickets**).
- **customer_id (FK):** Chave estrangeira que identifica o cliente comprador (liga à tabela **customers**).
- **sale_date:** Carimbo de data e hora (timestamp) do momento em que o pagamento/processamento foi concluído com sucesso.

3.5. Tabelas Adicionais do Sistema (**Loja e Autenticação**)

Embora o foco central exigido neste caso prático seja o ecossistema de bilheteira, a base de dados foi expandida de forma modular para suportar a plataforma completa da banda. Abaixo detalha-se o propósito das tabelas extra que gerem os utilizadores e a loja online:

Tabela **users Propósito:** Gere a autenticação e os perfis dos fãs registados na plataforma. É a tabela "mãe" para clientes logados.

- **id (PK):** Identificador único do utilizador no sistema.
- **username:** Nome de utilizador (ou nome próprio) escolhido para apresentar no perfil.
- **email:** Endereço de correio eletrónico associado à conta, usado para login e comunicações.
- **password:** Palavra-passe do utilizador (armazenada em formato de hash por motivos de segurança).

Tabela **products Propósito:** Funciona como o catálogo de merchandising oficial da banda (T-shirts, CDs, etc.) disponível na loja online.

- **id (PK):** Identificador único do artigo/produto.
- **name:** Designação do produto em catálogo.
- **price:** Valor de venda ao público do artigo físico.
- **image:** Referência ou caminho do ficheiro da imagem para visualização na interface da loja.

Tabela **orders Propósito:** Tabela mestre que regista cada transação feita na loja de merchandising (produtos físicos), separando-a das transações de bilheteira (**sales**).

- **id (PK)**: Identificador único da encomenda.
- **user_id (FK)**: Chave estrangeira que identifica qual o utilizador/fã que realizou a compra (relaciona-se com a tabela **users**).

Tabela **order_items Propósito:** Tabela de detalhe (pivot) que resolve a relação de muitos-para-muitos entre encomendas e produtos. Permite que uma única encomenda tenha múltiplos artigos diferentes.

- **id (PK)**: Identificador único da linha de encomenda.
- **order_id (FK)**: Chave estrangeira que associa esta linha à sua encomenda mestre (liga à tabela **orders**).
- **product_id (FK)**: Chave estrangeira que indica o artigo específico que foi adicionado ao carrinho (liga à tabela **products**).
- **quantity**: Regista a quantidade comprada deste produto específico dentro da encomenda.

4. Lógica das Views (Vistas de Dados)

Para simplificar a consulta de dados e otimizar o desempenho do painel de administração, foram criadas quatro Views. Estas vistas encapsulam consultas complexas que recorrem a agregações (`COUNT`, `SUM`), condicionalismos (`CASE WHEN`) e junções (`JOIN`, `LEFT JOIN`), permitindo extrair relatórios em tempo real sem sobrecarregar a base de dados.

4.1. View: `event_details`

- **Propósito:** Fornecer um panorama geral e rápido sobre o estado de cada concerto.
- **Lógica Específica:** Utiliza `LEFT JOIN` para cruzar a tabela `events` com a tabela `tickets`. Emprega a função `COUNT(t.id)` para calcular o total de bilhetes emitidos e a cláusula `SUM(CASE WHEN...)` para contar dinamicamente quantos bilhetes estão com o status 'Sold' (Vendidos) ou 'Available' (Disponíveis), apresentando esses totais em colunas separadas.

4.2. View: `customer_sales_summary`

- **Propósito:** Traçar o perfil de consumo de cada cliente/fã.
- **Lógica Específica:** Liga a tabela `customers` às tabelas `sales` e `tickets`. Usa a função `COUNT(s.id)` para saber o volume de bilhetes adquiridos por pessoa e `SUM(t.price)` com `IFNULL` para calcular a receita total gerada por esse cliente (garantindo que não aparecem valores nulos se o cliente ainda não tiver gasto nada).

4.3. View: `event_sales_summary`

- **Propósito:** Apurar a rentabilidade financeira de cada evento.
- **Lógica Específica:** Agrupa os dados pelo ID do evento (`GROUP BY e.id`). Soma o preço dos bilhetes (`SUM(t.price)`) apenas se estes constarem na tabela de vendas (`sales`). Esta view é crucial para a gestão financeira da digressão da banda.

4.4. View: `tickets_status`

- **Propósito:** Disponibilizar uma listagem limpa e descritiva do inventário de bilhetes.
- **Lógica Específica:** Substitui o ID numérico do evento pelo seu nome real através de um `INNER JOIN` simples entre `tickets` e `events`.

Isto facilita a leitura dos dados por parte de um administrador não técnico, mostrando o nome do concerto, lugar, preço e o estado atual.